

CS3.301: Operating Systems and Networks

Lecture 14 & 15 – Paging and TLBs

INSTRUCTOR: DEEPAK GANGADHARAN

Agenda

- **Part 1: Paging — Introduction**

- Pages and page frames, the page table, translating an address
- What a page-table entry holds, and why paging is slow and big

- **Part 2: Faster Translations — TLBs**

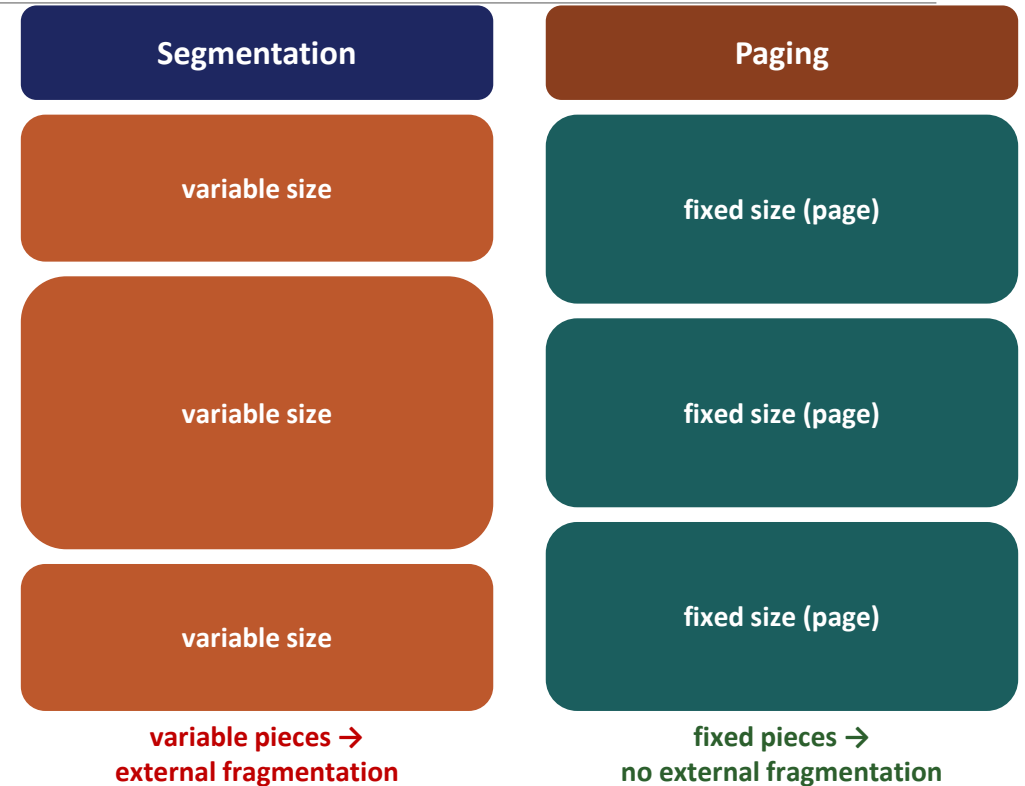
- The TLB as a translation cache: hits, misses and locality
- Who handles a miss, context switches, replacement, a real TLB

Recap: Two Ways to Manage Space

- OS space-management problems are usually solved in one of two ways
- Way 1: chop space into variable-sized pieces — this is what segmentation does
- Problem: different-size chunks fragment the space, so allocation gets harder over time
- Way 2: chop space into fixed-sized pieces — this idea is called paging
- Fixed-size units make placement simple and avoid external fragmentation

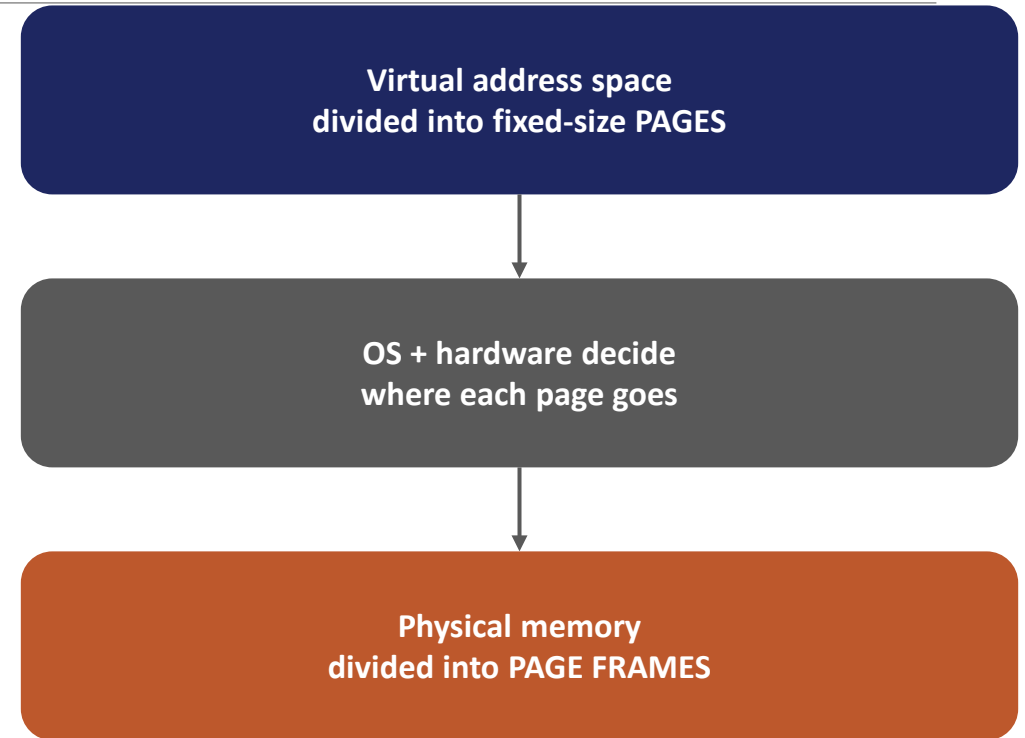
Paging: The Big Idea

- Divide a process's address space into fixed-sized units called pages
- View physical memory as an array of fixed-sized slots called page frames
- Each page frame holds exactly one virtual-memory page
- No logical code / heap / stack segments — just equal-sized pages
- The idea goes back to the Atlas system of the early 1960s



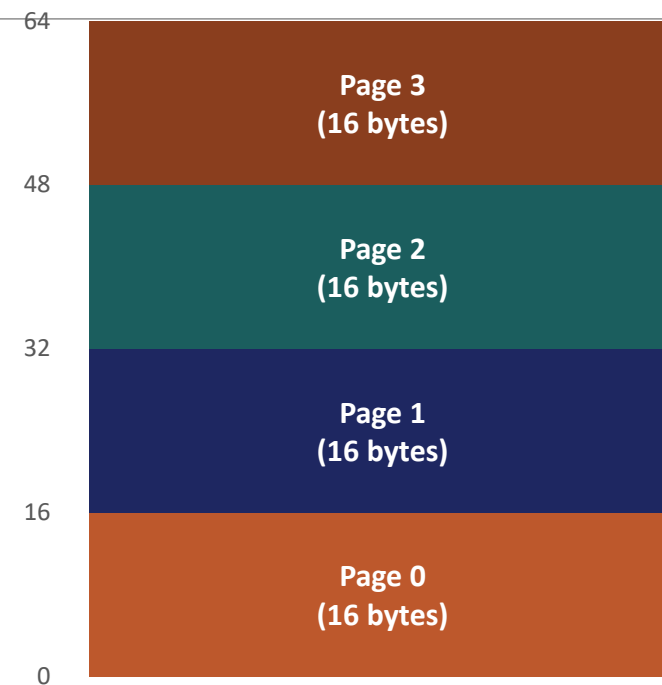
The Crux: Virtualizing Memory with Pages

- How can we virtualize memory with pages so as to avoid the problems of segmentation?
- What are the basic techniques?
- How do we make them work well, with minimal space and time overheads?
- First: the mechanism (page tables and translation)
- Then: making it fast (TLBs)



A Simple Example: A 64-Byte Address Space

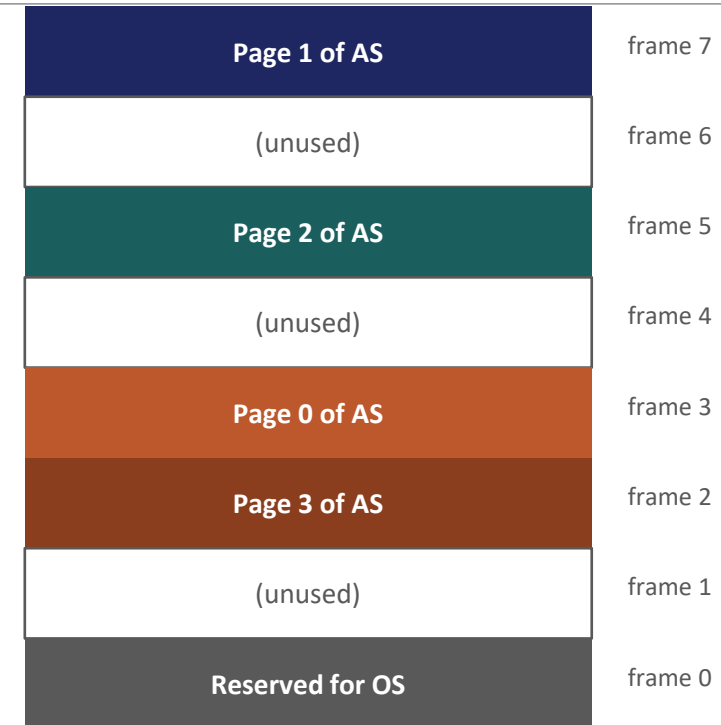
- A tiny address space of only 64 bytes, divided into four 16-byte pages
- Virtual pages 0, 1, 2 and 3
- Real address spaces are much larger: commonly 32 bits (4 GB) or 64 bits
- Tiny examples keep the arithmetic easy to follow



Virtual address space: 64 bytes
= 4 pages × 16 bytes

Physical Memory: Eight Page Frames

- Physical memory is also divided into 16-byte slots: eight page frames (128 bytes)
- Frame 0 is reserved for the OS
- VP 0 → frame 3, VP 1 → frame 7, VP 2 → frame 5, VP 3 → frame 2
- Frames 1, 4 and 6 are currently free
- Pages of the address space end up scattered around physical memory



128-byte physical memory = 8 frames × 16 bytes

Advantages of Paging

- **Flexibility**

- Supports the address-space abstraction however the process uses it
- No assumptions about which way the heap and stack grow

- **Simple free-space management**

- The OS keeps a free list of page frames
- To place a 64-byte address space, grab any four free frames

- **No external fragmentation: every unit is the same size**

The Page Table

- A per-process data structure recording where each virtual page lives in physical memory
- Stores one address translation per virtual page
- Our example: (VP 0 $\rightarrow$ PF 3), (VP 1 $\rightarrow$ PF 7), (VP 2 $\rightarrow$ PF 5), (VP 3 $\rightarrow$ PF 2)
- Another process has its own page table: its pages map to different frames
- Exception: inverted page tables, which are shared system-wide

Virtual page	Physical frame
VPN 0	PFN 3
VPN 1	PFN 7
VPN 2	PFN 5
VPN 3	PFN 2

One page table per process:
another process gets different mappings

Splitting a Virtual Address: VPN + Offset

- To translate an address, first split it into two components
- Virtual page number (VPN): selects the page
- Offset: selects the byte within that page
- 64-byte address space $\rightarrow$ 6-bit virtual addresses ($2^6 = 64$)
- 16-byte pages $\rightarrow$ 4 offset bits; the top 2 bits form the VPN

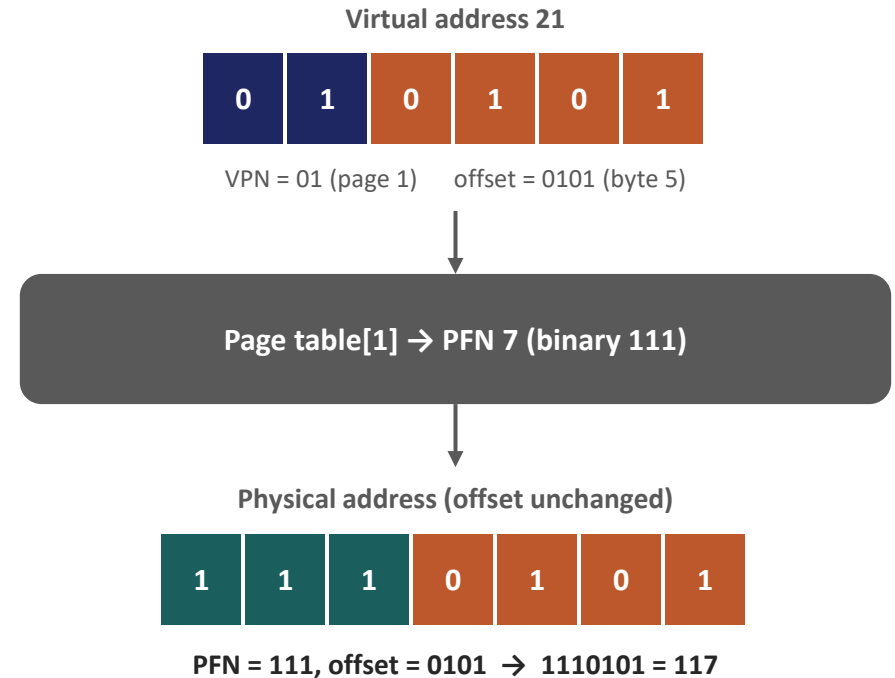


6-bit virtual address ($2^6 = 64$ bytes)

Top 2 bits $\rightarrow$ VPN: which of the 4 pages
Low 4 bits $\rightarrow$ offset: which of the 16 bytes

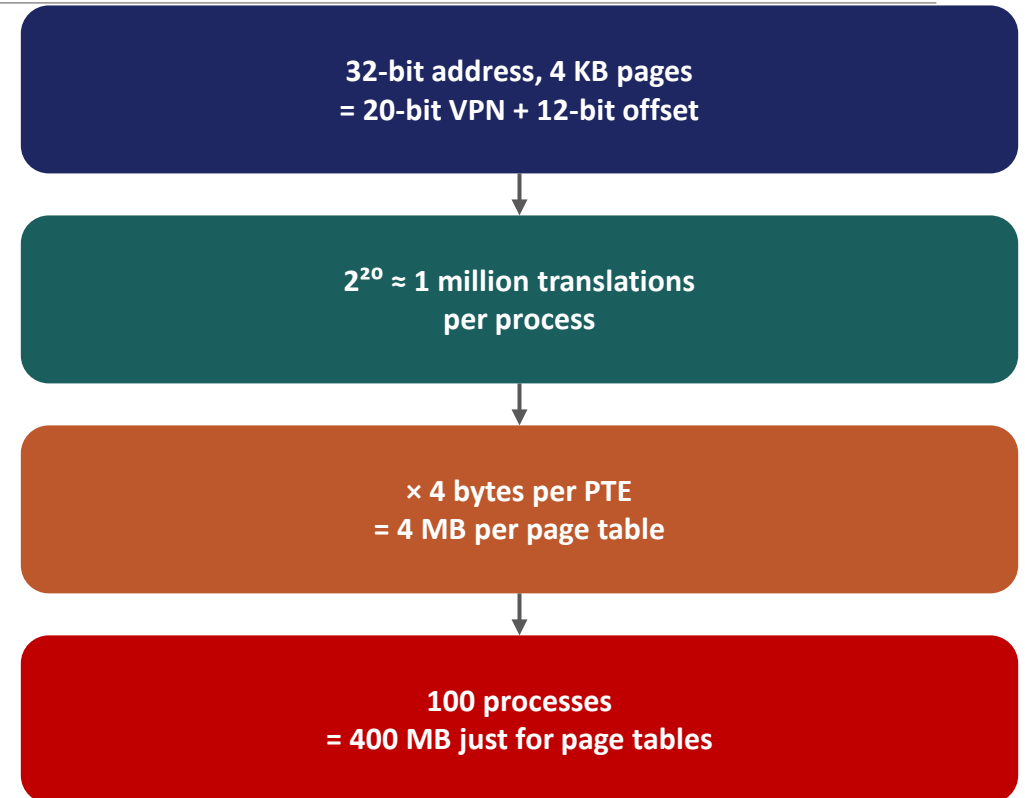
Worked Example: Translating Address 21

- Instruction: `movl 21, %eax` — virtual address 21 = 010101 in binary
- VPN = 01 (page 1); offset = 0101 (byte 5 of the page)
- The page table says VPN 1 → PFN 7 (binary 111)
- Replace the VPN with the PFN; the offset is NOT translated
- Physical address = 1110101 = 117



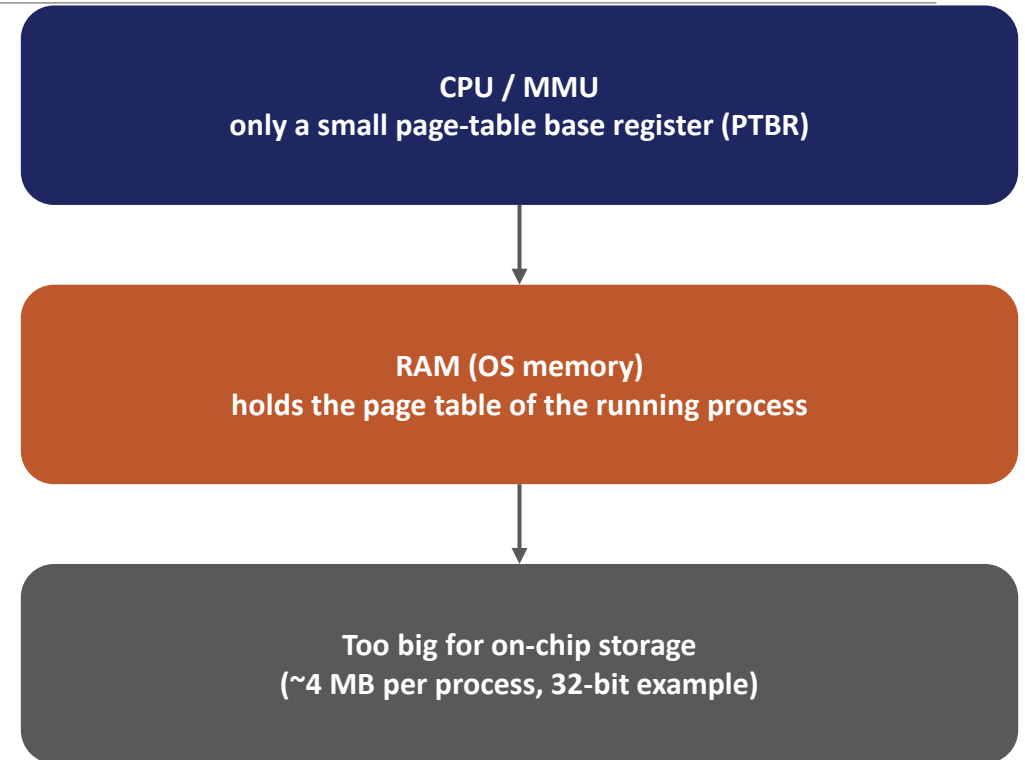
How Big Are Page Tables?

- Page tables can get terribly large — far bigger than a segment table or base/bounds pair
- Example: 32-bit address space with 4 KB pages → 20-bit VPN + 12-bit offset
- 2^{20} (about a million) translations per process
- At 4 bytes per page-table entry: 4 MB for each page table
- With 100 running processes: 400 MB just for address translations



Where Do Page Tables Live?

- Too big for special on-chip storage in the MMU
- So each process's page table is stored in memory
- For now: assume it sits in physical memory managed by the OS
- Later, page tables can themselves be virtualized or even swapped to disk
- The hardware only needs to know where the current table starts



The Linear Page Table

- A page table is just a data structure mapping VPNs to PFNs — any structure could work
- Simplest form: the linear page table, which is just an array
- The OS indexes the array by the virtual page number
- The entry found there is the page-table entry (PTE), which holds the PFN
- More advanced structures come in later lectures

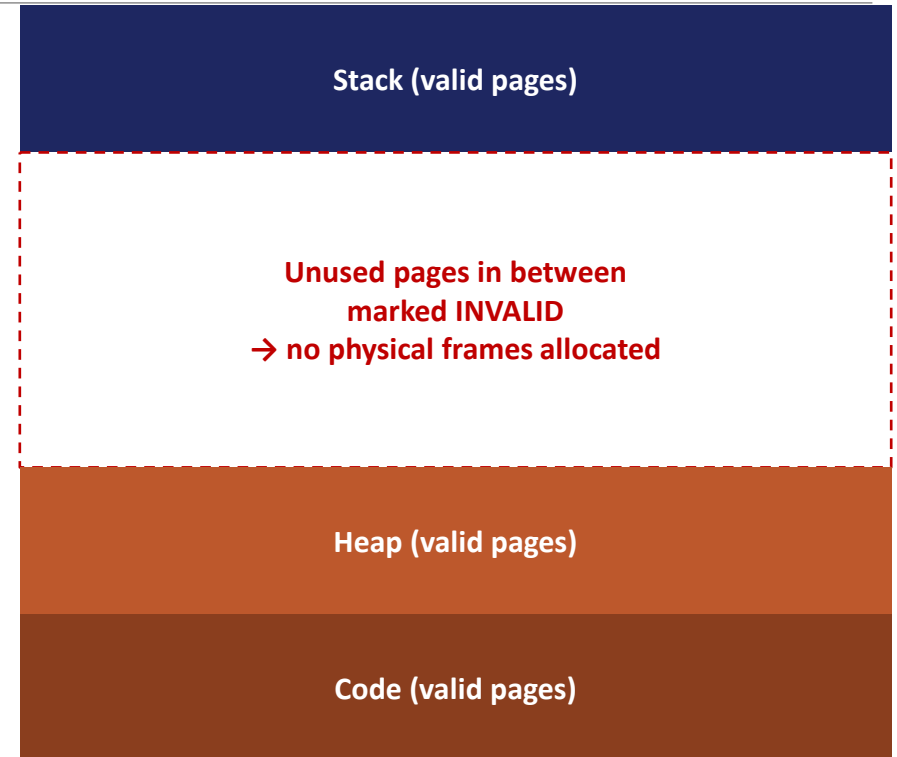
Linear page table = an array indexed by VPN

index 0	index 1	index 2	index 3
PTE PFN 3	PTE PFN 7	PTE PFN 5	PTE PFN 2

1. Use the VPN as the array index
2. Read the PTE stored at that index
3. Take the PFN (and check the bits) from the PTE

PTE Bits: The Valid Bit

- A valid bit says whether this particular translation is valid
- A new process has code and heap at one end of its address space, the stack at the other
- All the unused space in between is marked invalid
- Touching an invalid page traps to the OS, which will likely terminate the process
- The valid bit is crucial for sparse address spaces: no frames are needed for invalid pages



Access to an invalid page → trap to the OS

Other Bits in a Page Table Entry

- Protection bits: may the page be read, written, or executed from?
- Present bit: is the page in physical memory or swapped out to disk?
- Dirty bit: has the page been modified since it was brought into memory?
- Reference (accessed) bit: tracks which pages are popular — useful for page replacement
- Accessing a page in a way the bits disallow traps to the OS

Valid

is this page allocated at all?

Protection

read / write / execute allowed?

Present

in physical memory or swapped to disk?

Dirty

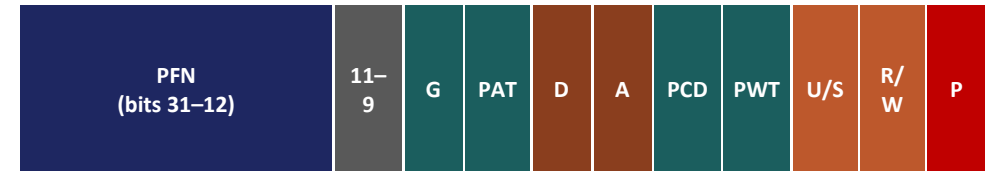
modified since it was brought in?

Reference

accessed recently? (popular page)

Example: The x86 Page Table Entry

- Present (P), read/write (R/W) and user/supervisor (U/S) bits
- PWT, PCD, PAT and G control how hardware caching works for the page
- Accessed (A) and dirty (D) bits
- The page frame number (PFN) occupies bits 31–12
- No separate valid bit: P=1 means present and valid; P=0 traps and the OS decides



P: present

R/W: writes allowed?

U/S: user-mode access allowed?

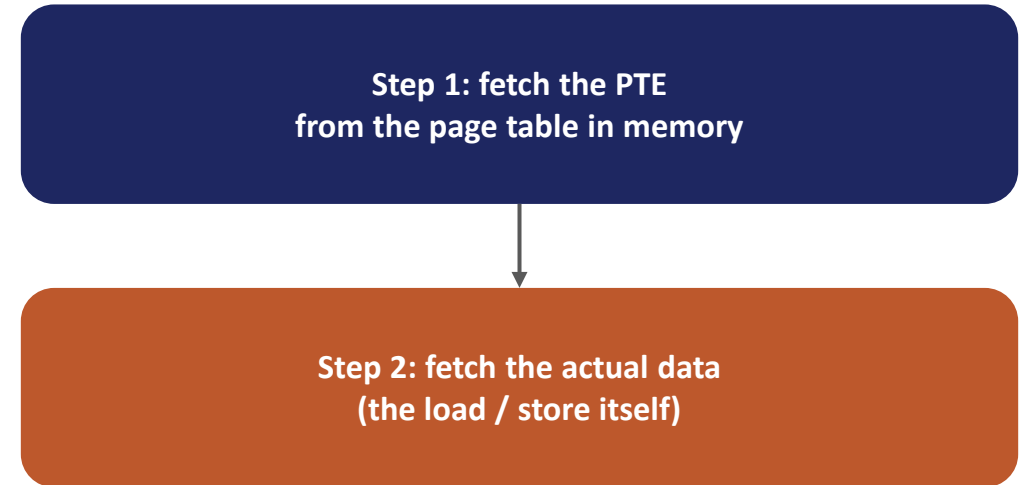
PWT, PCD, PAT, G: hardware caching behavior

A: accessed bit D: dirty bit

PFN: the physical frame number itself

Paging Is Also Too Slow

- Page tables live in memory, so translation itself needs a memory access
- Before fetching the data at address 21, hardware must first fetch the PTE
- Hardware must know where the page table is: a page-table base register (PTBR)
- Every memory reference (fetch, load, store) costs one extra memory reference
- **Result: a slowdown of a factor of two or more**



**Two memory references instead of one:
roughly a 2× slowdown or worse**

Hardware Steps: Virtual to Physical Address

- Extract the VPN using a mask and a shift
- Compute the PTE address from the page-table base register
- Fetch the PTE from memory and read its PFN
- Shift the PFN and OR in the offset to form the physical address
- Example: VPN_MASK = 0x30 and SHIFT = 4

translation.c

```
// Extract the VPN
VPN = (VirtualAddress & VPN_MASK) >> SHIFT

// Address of the page-table entry
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE))

// After the PTE has been fetched
offset = VirtualAddress & OFFSET_MASK
PhysAddr = (PFN << SHIFT) | offset
```

Memory Access Protocol with Paging

- Every memory reference follows this protocol
- Extract the VPN and compute the PTE address
- Fetch the PTE from memory (the extra reference)
- Check the valid bit and the protection bits
- Form the physical address and access memory

```
paging_access.c

// Extract the VPN from the virtual address
VPN = (VirtualAddress & VPN_MASK) >> SHIFT

// Form the address of the page-table entry (PTE)
PTEAddr = PTBR + (VPN * sizeof(PTE))

// Fetch the PTE
PTE = AccessMemory(PTEAddr)

// Check if process can access the page
if (PTE.Valid == False)
    RaiseException(SEGMENTATION_FAULT)
else if (CanAccess(PTE.ProtectBits) == False)
    RaiseException(PROTECTION_FAULT)
else
    // Access OK: form physical address and fetch it
    offset = VirtualAddress & OFFSET_MASK
    PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
    Register = AccessMemory(PhysAddr)
```

A Memory Trace: The array.c Program

- A C loop that zeroes an array of 1000 integers
- Compiles to four instructions at 1024–1036 (assume 4 bytes each)
- `movl` stores 0 at `%edi + %eax*4`: `%edi` is the array base, `%eax` the index
- `incl` bumps the index; `cmpl` compares with 1000 (0x3e8); `jne` loops back
- Question: which memory accesses does this loop make?

```
array.c

int array[1000];
...
for (i = 0; i < 1000; i++)
    array[i] = 0;

// disassembled loop (objdump / otool)
1024 movl $0x0, (%edi,%eax,4)
1028 incl %eax
1032 cmpl $0x03e8,%eax
1036 jne 1024
```

Trace Assumptions: Page Table and Mappings

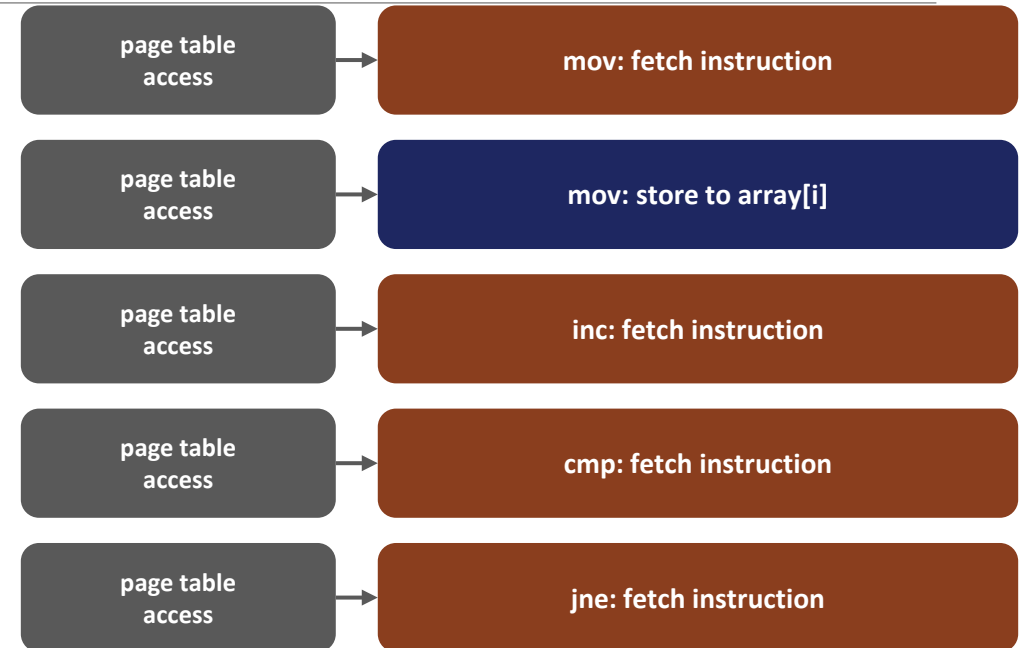
- Virtual address space of 64 KB (unrealistically small)
- Page size of 1 KB
- Linear page table located at physical address 1 KB (1024)
- Code at virtual address 1024 → VPN 1 → PFN 4
- Array at virtual addresses 40000–44000 → VPNs 39–42 → PFNs 7, 8, 9, 10

64 KB address space · 1 KB pages
Linear page table at physical address 1 KB (1024)

Region	Virtual page	Physical frame
Code	VPN 1	PFN 4
Array	VPN 39	PFN 7
Array	VPN 40	PFN 8
Array	VPN 41	PFN 9
Array	VPN 42	PFN 10

Memory Accesses per Loop Iteration

- Each instruction fetch needs a page-table access first, then the fetch itself
- The explicit store to the array also needs a page-table access, then the store
- Per iteration: 4 instruction fetches + 1 array update + 5 page-table accesses
- That is 10 memory accesses for what looks like 4 instructions
- Patterns change as the loop moves onto new array pages



**5 page-table accesses + 5 real accesses
= 10 memory references per iteration**

Summary: Paging So Far

- Paging chops the address space into fixed-size pages and memory into page frames
- Advantages: no external fragmentation, and flexible sparse address spaces
- Problem 1: an extra memory access on every reference
- Problem 2: page tables occupy a lot of memory
- Next: hardware caching (the TLB) attacks the speed problem

Pros

No external fragmentation

Flexible: supports sparse address spaces

Cons

Extra memory access on every reference

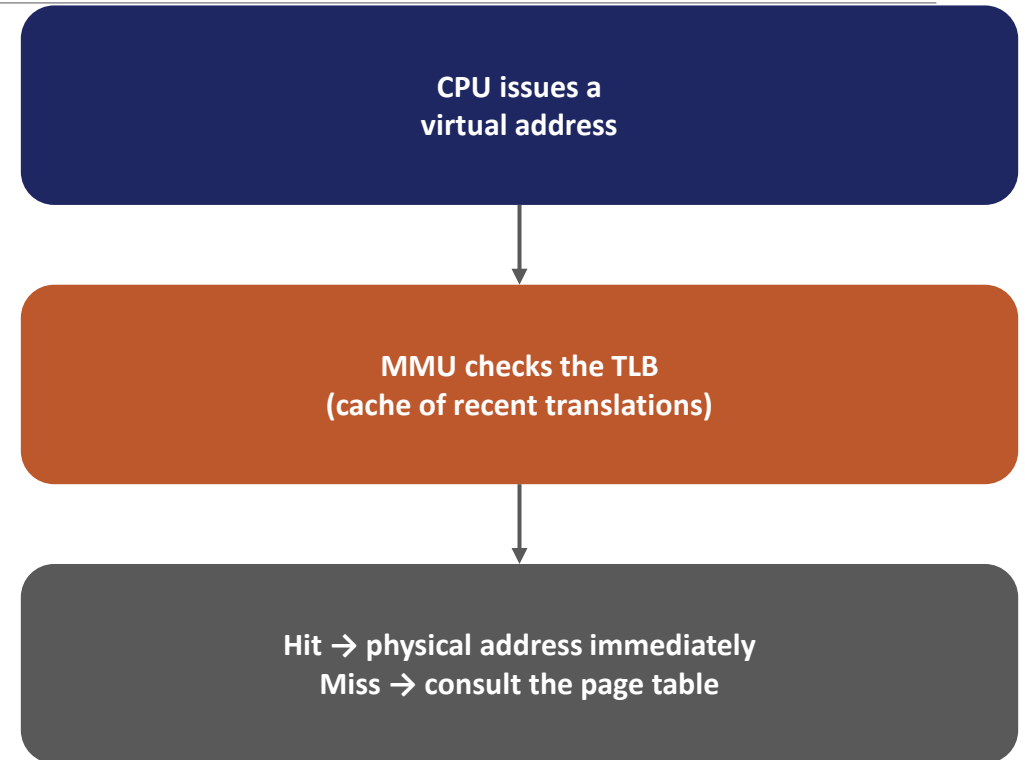
Page tables consume lots of memory

The Crux: How to Speed Up Translation

- Paging needs an extra memory lookup for every virtual address the program generates
- Going to memory for translation before every fetch, load or store is prohibitively slow
- **How can we speed up address translation and avoid the extra reference?**
- What hardware support is required?
- What OS involvement is needed?

The TLB: An Address-Translation Cache

- TLB = translation-lookaside buffer, part of the chip's memory-management unit (MMU)
- A hardware cache of popular virtual-to-physical address translations
- On every virtual memory reference, hardware checks the TLB first
- If the translation is there, the page table is not consulted
- “Address-translation cache” would be a better name; TLBs make virtual memory practical



TLB Basic Algorithm

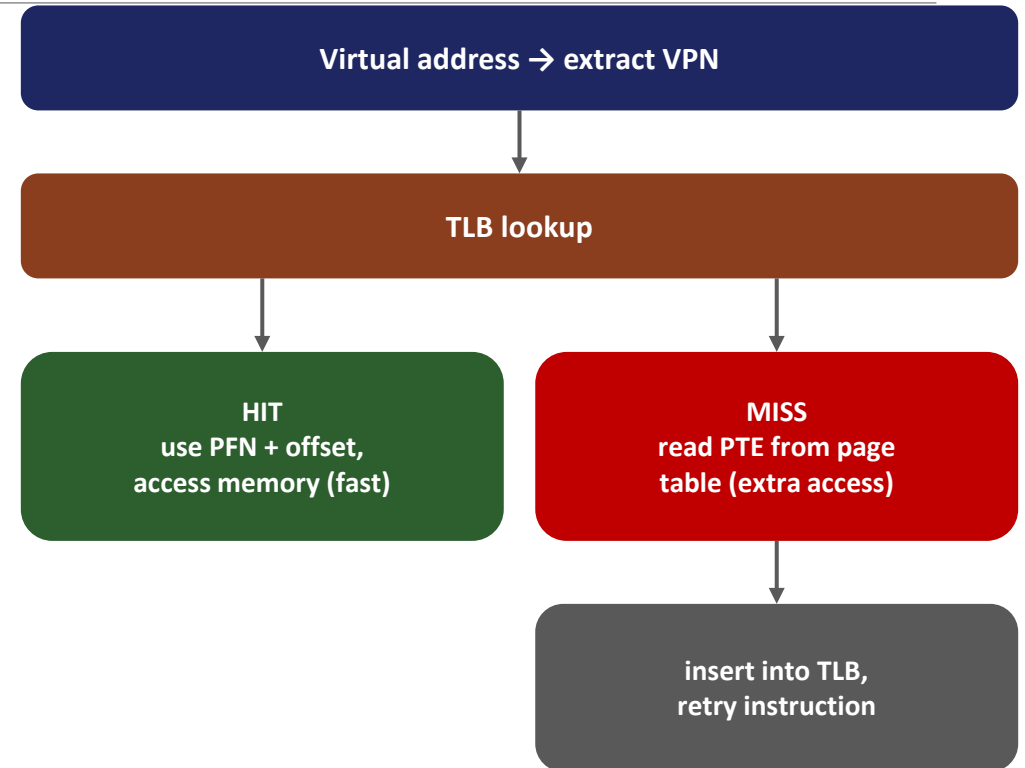
- Extract the VPN and look it up in the TLB
- Hit: check protection, form the physical address, access memory
- Miss: read the PTE from the page table (the extra memory reference)
- If valid and accessible: insert it into the TLB and retry the instruction
- The retry now hits in the TLB

tlb_control_flow.c

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
(Success, TlbEntry) = TLB_Lookup(VPN)
if (Success == True) // TLB Hit
    if (CanAccess(TlbEntry.ProtectBits) == True)
        Offset = VirtualAddress & OFFSET_MASK
        PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
        Register = AccessMemory(PhysAddr)
    else
        RaiseException(PROTECTION_FAULT)
else // TLB Miss
    PTEAddr = PTBR + (VPN * sizeof(PTE))
    PTE = AccessMemory(PTEAddr)
    if (PTE.Valid == False)
        RaiseException(SEGMENTATION_FAULT)
    else if (CanAccess(PTE.ProtectBits) == False)
        RaiseException(PROTECTION_FAULT)
    else
        TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
        RetryInstruction()
```

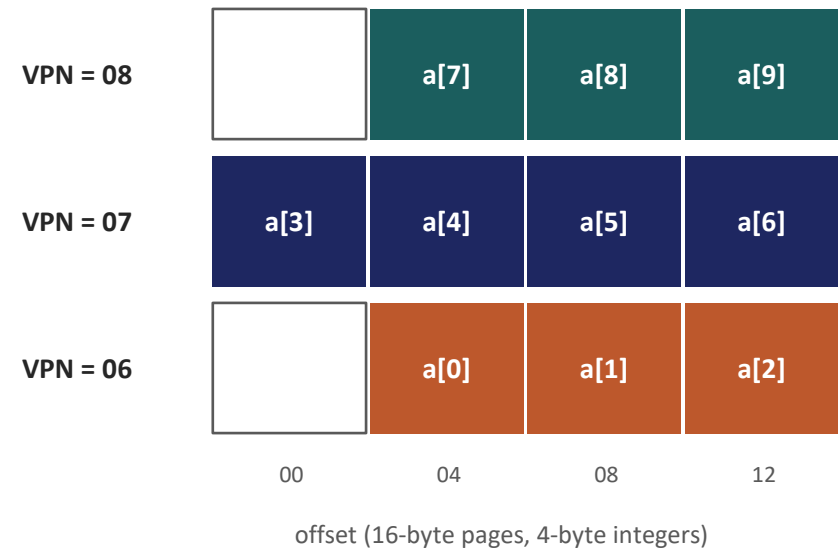
TLB Hit vs. TLB Miss

- Hit: the translation is already cached, so the access proceeds quickly
- Miss: we pay the full cost of paging — an extra memory reference to the page table
- TLBs work because, in the common case, translations are hits
- The TLB sits near the processing core and is designed to be very fast
- If misses are frequent, the program runs noticeably slower



Example: Accessing an Array

- Array of 10 four-byte integers starting at virtual address 100
- 8-bit virtual addresses, 16-byte pages: 4-bit VPN + 4-bit offset
- $a[0]$ is at VPN 6, offset 4; only three integers fit on that page
- $a[3]$ – $a[6]$ are on VPN 7; $a[7]$ – $a[9]$ are on VPN 8
- Loop: $\text{sum} += a[i]$ for $i = 0 \dots 9$ (ignore i , sum and instruction fetches)



$a[0]$ is at virtual address 100:
VPN 6, offset 4. The array spans
three pages (VPN 6, 7, 8).

TLB Activity Over Ten Accesses

- a[0]: miss — first touch of VPN 6; a[1], a[2]: hits
- a[3]: miss — first touch of VPN 7; a[4]–a[6]: hits
- a[7]: miss — first touch of VPN 8; a[8], a[9]: hits
- Pattern: miss, hit, hit, miss, hit, hit, hit, miss, hit, hit
- Hit rate = $7/10 = 70\%$ even on the first pass over the data

First pass over the array

a0	a1	a2	a3	a4	a5	a6	a7	a8	a9
miss	hit	hit	miss	hit	hit	hit	miss	hit	hit

7 hits / 10 accesses = 70% hit rate

Second pass (translations already cached)

hit	hit	hit	hit	hit	hit	hit	hit	hit	hit
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

10 hits / 10 accesses → temporal locality

Locality: Why TLBs Work

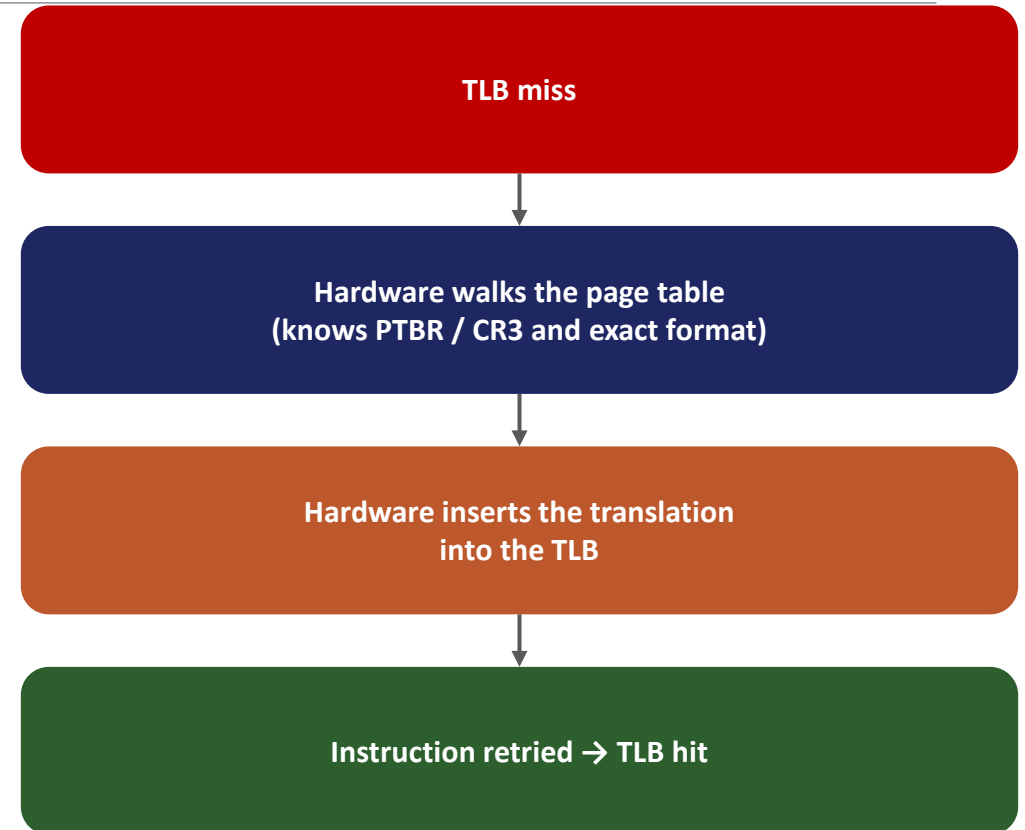
- Caching keeps copies of memory in small, fast, on-chip storage
- **Spatial locality:** if x is accessed, addresses near x will likely be accessed soon
- **Temporal locality:** recently used items will likely be used again soon
- TLBs rely on both — these are properties of the program, not hard laws
- Typical pages are 4 KB, so dense array access gives about one miss per page

Spatial locality
if x is accessed, addresses near x
will likely be accessed soon
(array elements share a page)

Temporal locality
recently accessed items
will likely be re-accessed soon
(loops, second pass over an array)

Hardware-Managed TLBs

- Who handles a TLB miss? Two possibilities: the hardware or the OS
- Older CISC hardware handled misses entirely, because designers did not trust the OS
- Hardware must know where the page table is (PTBR) and its exact format
- On a miss: walk the page table, extract the translation, update the TLB, retry
- Example: Intel x86, with a fixed multi-level page table; CR3 points to the current one



Software-Managed TLBs

- Modern RISC architectures (MIPS, SPARC v9) use software-managed TLBs
- On a miss, hardware just raises an exception and traps to the OS
- The trap handler is OS code written expressly to handle TLB misses
- It looks up the translation, updates the TLB with privileged instructions, returns from the trap
- Advantages: flexibility (any page-table structure) and simple hardware

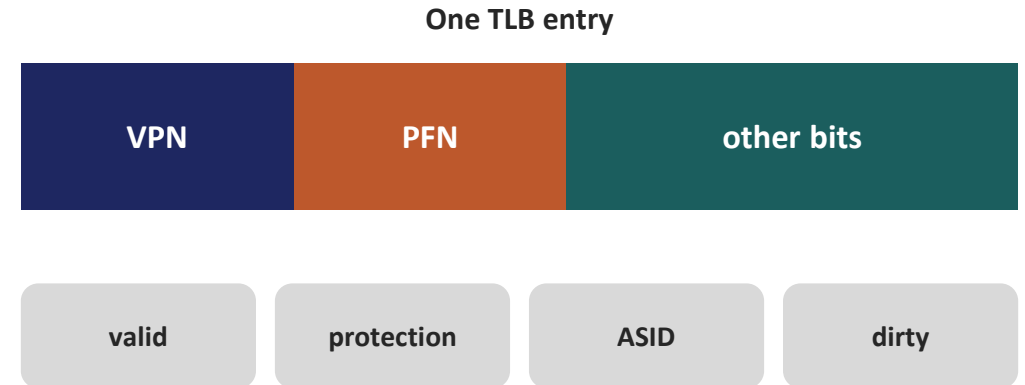


Software-Managed TLBs: Subtle Details

- Return-from-trap must resume at the instruction that missed (retry), not the next one
- So hardware saves a different PC depending on what caused the trap
- The miss handler must not itself cause an endless chain of TLB misses
- Fix 1: keep handler code in physical memory (unmapped, so no translation needed)
- Fix 2: reserve wired TLB entries with permanent translations that always hit

What Is Inside a TLB Entry?

- Both VPN and PFN are stored: a translation may sit in any slot (fully associative)
- Valid bit: does this entry hold a valid translation?
- TLB valid bit $\neq$ page-table valid bit: an invalid TLB entry is just empty (e.g., after boot)
- An invalid PTE means the page was never allocated to the process
- Also common: protection bits, an address-space identifier (ASID) and a dirty bit



Typical TLB: 32, 64 or 128 entries,
fully associative — any translation can sit
in any slot; hardware compares all
entries in parallel

Context Switches: The Problem

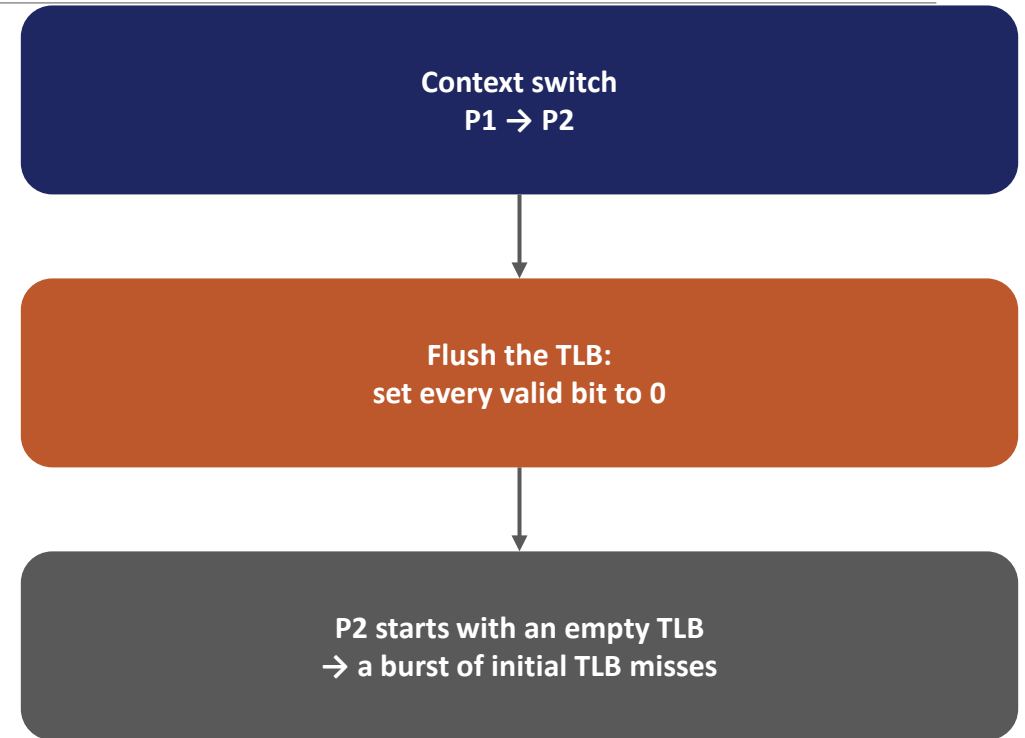
- TLB translations are only valid for the currently running process
- P1's virtual page 10 → frame 100; P2's virtual page 10 → frame 170
- If both entries are in the TLB, hardware cannot tell which one is meant
- The about-to-run process must never use the previous process's translations
- Crux: how should TLB contents be managed on a context switch?**

VPN	PFN	valid	prot
10	100	1	rwX
—	—	0	—
10	170	1	rwX
—	—	0	—

Entry 1: P1's VPN 10 → PFN 100
Entry 3: P2's VPN 10 → PFN 170
Which one should the hardware use?

Solution 1: Flush the TLB

- Simplest approach: empty the TLB on every context switch
- Software-managed TLB: an explicit privileged flush instruction
- Hardware-managed TLB: flush whenever the page-table base register changes
- A flush just sets all valid bits to 0
- **Cost: every run starts with TLB misses — expensive if switches are frequent**



Solution 2: Address Space Identifiers

- Add an address-space identifier (ASID) field to each TLB entry
- Think of it as a small process ID (e.g., 8 bits instead of a 32-bit PID)
- Entries from different processes can coexist in the TLB
- On a context switch the OS sets a privileged register to the current ASID
- No flush needed — avoids the burst of misses after each switch

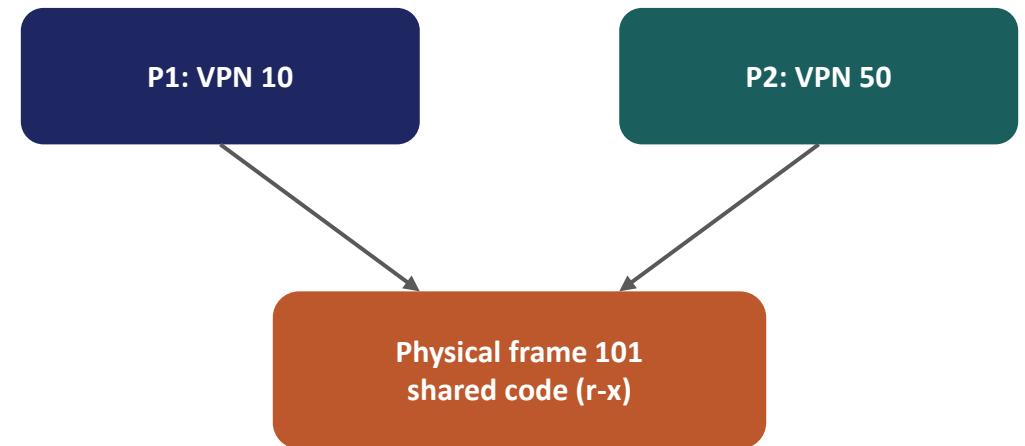
VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
—	—	0	—	—
10	170	1	rwX	2
—	—	0	—	—

Same VPN, different ASID → no confusion.
OS sets a privileged register to the
current process's ASID on a context switch.

Sharing Pages Across Processes

- Two TLB entries can look similar: different VPNs pointing to the same PFN
- Example: P1's VPN 10 and P2's VPN 50 both map to physical frame 101 (r-x)
- Happens when processes share a page, e.g., code in binaries or shared libraries
- Sharing reduces the number of physical pages in use
- Memory overhead goes down

VPN	PFN	valid	prot	ASID
10	101	1	r-x	1
50	101	1	r-x	2



TLB Replacement Policy: LRU vs. Random

- Installing a new TLB entry means replacing an old one — which one?
- LRU: evict the least-recently-used entry, exploiting locality
- Random: evict a random entry — simple, and avoids corner-case behavior
- Corner case: a loop over $n + 1$ pages with an n -entry TLB
- **LRU misses on every access there; random does much better**

TLB holds 3 entries; a loop touches 4 pages



Under LRU, the entry evicted is always the one needed next: EVERY access misses.

Random replacement avoids this corner case and does much better.

A Real TLB Entry: The MIPS R4000

- Software-managed TLB; 32-bit address space with 4 KB pages
- Only 19 VPN bits: user addresses come from half the space (the rest is the kernel's)
- Up to a 24-bit PFN → up to 64 GB of physical memory
- Global bit (G), 8-bit ASID, 3 coherence bits, dirty and valid bits
- Wired register reserves TLB slots for the OS; four privileged instructions manage entries

MIPS R4000 TLB entry (simplified)



G: global bit (ASID ignored)

C: cache coherence bits

D: dirty V: valid

OS instructions: TLBP (probe), TLBR (read),
TLBWI (write indexed), TLBWR (write random)

Culler's Law and TLB Coverage

- RAM isn't always RAM: accessing some pages costs more than others
- Touching more pages than the TLB can map (exceeding TLB coverage) causes many misses
- Measuring: touch more and more pages in a loop and time the accesses
- Jumps in access time reveal the TLB sizes — here, a two-level TLB
- Larger pages increase coverage (used by databases for big, randomly-accessed structures)

≤ 8 pages: ~5 ns (first-level TLB hits)



16+ pages: ~20 ns (second-level TLB)



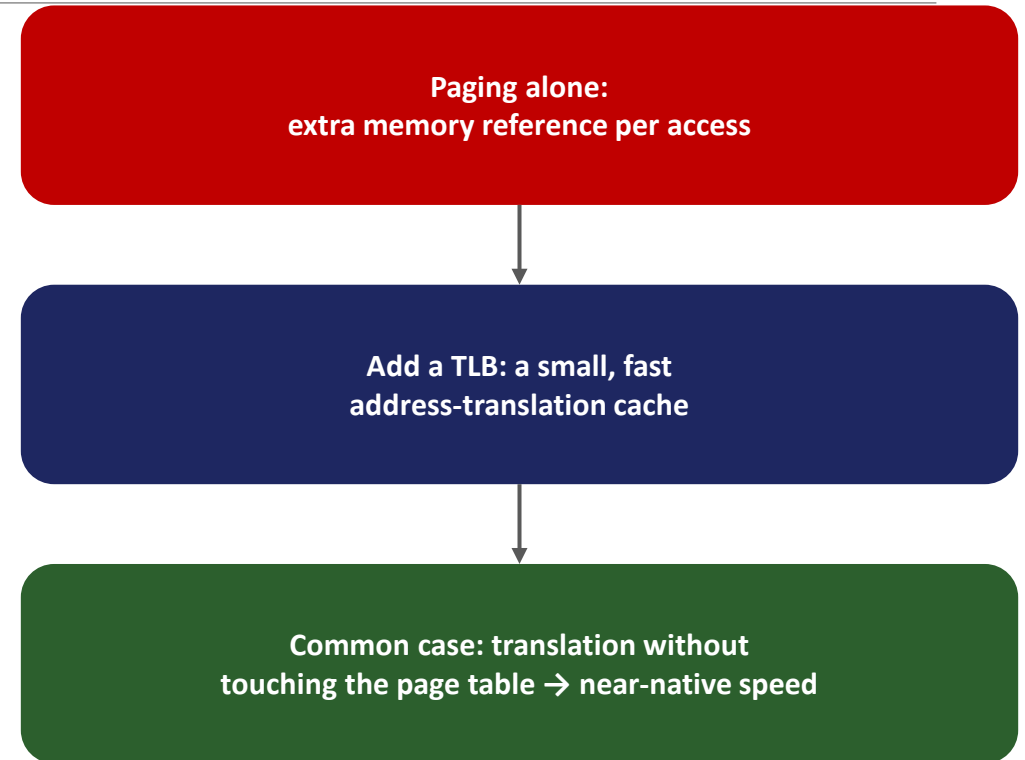
~1024+ pages: ~70 ns (page-table walks)



Sudden jumps reveal the TLB sizes.
Hit vs. miss: roughly a 14× difference.

Summary: TLBs

- A small on-chip TLB caches translations, so most references avoid the page table
- In the common case performance is nearly as if memory weren't virtualized
- Programs that touch more pages than fit in the TLB suffer many misses
- Large pages can increase the effective coverage of the TLB
- TLB access can bottleneck the CPU pipeline with physically-indexed caches; virtually-indexed caches help but add complexity



Thank You!